Laboratorio de Programación y Estructuras de Datos –
Resumen de lenguaje C

1. La estructura de un programa en C

La estructura general simplificada de un programa en C es la siguiente:

```
/* Incluir librerías */
#include <nom_lib.h>
#include "nom_lib.h"

/* Definir constantes */
#define NOM_CONS valor_cons

/* Programa principal */
main()
{
   /* Declaración de variables */
   tipo_var nom_var;

   /* Sentencias */
}
```

En primer lugar, debemos saber que en el lenguaje C, todo el texto escrito entre los símbolos `/*` y `*/` es un comentario y, por tanto, es ignorado por el compilador (no forma parte del programa).

Podemos diferenciar tres componentes en la estructura básica de un programa: incluir librerías, definir constantes y el programa principal.

Incluir librerías

Los compiladores de C proporcionan librerías de funciones. Cada librería tiene asociada un fichero de definición que se llama *cabecera* (*header* en inglés). Para utilizar cualquier función de la librería estándar de C, debemos incluir en nuestro programa el fichero de *cabecera* correspondiente de la siguiente manera:

```
#include <nom_lib.h>
```

donde `nom_lib.h` es el fichero de *cabecera* de la librería que se desea utilizar. Por ejemplo: `stdio.h` para la librería estándar de entrada/salida, `string.h` para la librería que manipula cadena de caracteres, `math.h` para la librería matemática, etc.

```
#include <stdio.h>
#include <string.h>
#include <math.h>
```

También podemos tener nuestras propias librerías. En este caso debemos tener un fichero de *cabecera* (con extensión `.h`) y un fichero de código (con extensión `.c`) que contiene la implementación de las funciones. Para utilizar las funciones de nuestra propia librería, debemos incluir el fichero de *cabecera* en el programa de la siguiente manera:

```
#include "nom_lib.h"
```

Nótese que en lugar de los símbolos < y > se usan las dobles comillas (″″) y que no se incluye el fichero con extensión `.c`. En este caso, el fichero de cabecera *`nom_lib`*`.h` y el fichero de código *`nom_lib`*`.c` deben estar almacenados en el mismo directorio que el programa que lo incluye.

Por ejemplo, si tuviéramos una librería propia con funciones que permiten escribir en colores por pantalla, entonces debemos tener dos ficheros: `colours.h` (la cabecera) y `colours.c` (fichero código con la implementación de las funciones). Si queremos escribir un programa que usa esta librería, debemos incluirla, escribiendo:

```
#include ″colours.h″
```

y al compilar el programa, debemos hacerlo junto con el fichero `colours.c`.

Definir constantes

Una constante en C es un identificador que representa un valor que no puede ser modificado durante la ejecución del programa. Para definir una constante, debemos escribir:

```
#define NOM_CONS valor_cons
```

Donde *`NOM_CONS`* es el nombre de la constate y *`valor_cons`* el valor que tiene. El nombre de la constante lo escribiremos siempre en mayúsculas. A continuación mostramos algunos ejemplos típicos:

```
#define PI 3.1415
#define CIERTO 1
#define FALSO 0
#define SI 'S'
```

Programa principal

Todos los programas tienen una función principal llamada `main` que indica el punto inicial de entrada. Es la función que se ejecuta al inicio. Dentro del programa principal se declaran las variables y se indica el orden de ejecución de las sentencias o acciones.

El programa principal está englobado entre llaves.

```
main()
{
   /* Declaración de variables */

   /* Sentencias */
}
```

Declaración de variables: Una variable es un identificador que guarda un único valor que puede ser consultado y modificado durante la ejecución de un programa. Se pueden declarar de las siguientes maneras:

```
tipo_var nom_var1;
tipo_var nom_var2, nom_var3;
tipo_var nom_var4=valor_inicial;
```

donde *tipo_var* indica el tipo de dato de la variable, *nom_varX* (con X=1,2,3,4) el nombre de esta y *valor_inicial* el valor que le damos a la variable al inicio del programa. De momento trabajaremos únicamente con 3 tipos de datos: `int` (para almacenar un valor entero), `char` (para almacenar un carácter) y `float` (para almacenar un valor real). El nombre de la variable siempre lo escribiremos en minúsculas.

A continuación se muestran algunos ejemplos:

```
int valor;          /* declara valor como entero, sin inicializar */
char letra='F';     /* declara letra como carácter, inicializada a
                       'F' */
float area1,area2;  /* declara area1 y area2 como reales, sin
                       inicializar */
```

***Sentencias*:** Una sentencia es una acción. De momento vamos a introducir únicamente tres tipos de sentencias: asignación, llamada a la función `printf()` y llamada a la función `scanf()`.

(a) La sentencia de asignación asigna un valor a una variable:

```
nom_var=expresion;  /* asigna a la variable nom_var el resultado
                       de la expresión */
```

Algunos ejemplos de sentencias de asignación son los siguientes:

```
valor=8*4;    /* asigna a la variable valor el valor 32 */
letra='M';    /* asigna a la variable letra el carácter 'M' */
area1=PI*valor*valor; /* asigna a la variable area1 el resultado
                         de la expresión PI*valor*valor, es
                         decir, 3215.36 (3.14*32*32)
```

(b) La llamada a la función `printf()` nos permitirá mostrar mensajes por pantalla. Para usar esta función, debemos escribir:

```
printf("Texto que queremos mostrar por pantalla", param1, …);
```

Entre comillas dobles se indica el texto que se mostrará por pantalla y a continuación, separados por comas, las variables o expresiones (*param1,…*) cuyos valores también se desean mostrar por pantalla. El texto puede incluir los especificadores de formato `%d` (para valores de tipo `int`), `%c` (para valores de tipo `char`) o `%f` (para valores de tipo `float`) dentro de las comillas dobles. El `printf()` sustituye cada especificador de formato por el valor de la variable o expresión correspondiente (*param1,…*).

Algunos ejemplos de uso de la función `printf()` son los siguientes:

```
printf("Buenos dias\n");
printf("Variable valor: %d, Variable letra: %c\n", valor, letra);
printf("El area de la circunferencia es %.2f\n", area1);
```

Algunas consideraciones en el uso del `printf()`:

- No utilice acentos ni letras especiales (ñ, ç, etc) en el texto que se quiere mostrar en pantalla.
- El carácter especial `'\n'` indica salto de línea. En pantalla veremos que el cursor se sitúa en la siguiente línea.
- Otros caracteres especiales son: `'\t'` (tabulador), `'\\'` (para mostrar la `'\'` en pantalla) y `'%%'` (para mostrar el carácter `'%'` en pantalla).
- Para mostrar por pantalla el valor de una variable o expresión debemos usar el especificador de formato `%d` (si el valor es de tipo `int`), `%c` (si es de tipo `char`) o `%f` (si es de tipo `float`) dentro de las comillas dobles y añadir como parámetro la variable o expresión asociada al valor que se desea mostrar por pantalla. El `printf()` sustituirá el especificador de formato por el valor de la variable o expresión correspondiente. Podemos tener varios especificadores de formato en un único `printf()`:

```
printf("area1: %f \t area2: %f \t Suma: %.2f\n", area1,
        area2, area1+area2);
```

- El especificador `%f` muestra un valor real con seis decimales. Si queremos mostrar menos decimales podemos usar `%.2f` (2 decimales), `%.3f` (3 decimales), etc.

(c) La llamada a la función `scanf()` nos permitirá leer datos introducidos desde teclado por el usuario del programa. Para usar esta función en su versión más simple, debemos escribir:

```
scanf("especificador_de_formato", &nom_var);
```

donde el *especificador_de_formato* es `%d` (si queremos leer un valor entero), `%c` (si queremos leer un carácter) o `%f` (si queremos leer un valor real) y *nom_var* es el nombre de la variable donde se guardará el valor leído del teclado. Obviamente, esta variable debe ser declarada del tipo correspondiente al inicio del programa.

Algunos ejemplos de uso de la función `scanf()` son los siguientes:

```
scanf("%d", &valor);
scanf("%f %f%", &area1, &area2);
```

Algunas consideraciones en el uso del `scanf()`:

- La función `scanf()` no muestra nada por pantalla. Si queremos pedir al usuario que introduzca un valor entero debemos hacerlo con dos sentencias:

```
printf("Introduzca un valor entero:");
scanf("%d", &valor);
```

- Los especificadores de formato `%d` y `%f` ignoran los caracteres espacios, tabuladores y/o saltos de líneas que pueda haber delante del número. Sin embargo, el especificador de formato `%c` no los ignora. Por ejemplo, si ejecutamos el siguiente código:

```
printf("Introduzca un valor entero y otro real: ");
scanf("%d%f", &valor, &area1);
printf("Introduzca un caracter: ");
scanf("%c", &letra);
```

y la entrada que introduce el usuario (marcada en negrita) es:

```
Introduzca un valor entero y otro real: 15 3.6
Introduzca un caracter:
```

en la variable `valor` quedará almacenado el valor `15` y en `area1` el `3.6`. Sin embargo, el programa no esperará a que el usuario introduzca un carácter. En su lugar, en la variable `letra` quedará almacenado un `'\n'` (salto de línea) y el programa finalizará. Esto se debe a que después de introducir el valor `3.6`, el usuario ha pulsado la tecla <ENTER> o salto de línea (que en pantalla no se visualiza). Cuando se ejecuta la sentencia `scanf("%c", &letra)` se lee el carácter `'\n'` (salto de línea) de la entrada y se almacena en `letra`.

- Para solventar el problema anterior, se usa el especificador de formato `%*c` que lee del teclado un carácter y lo descarta, es decir, no lo almacena en ninguna variable. Este especificador de formato se usa para el leer el salto de línea (`'\n'`) que pulsamos después de introducir los datos. El código correcto del ejemplo anterior es:

```
printf("Introduzca un valor entero y otro real: ");
scanf("%d%f%*c", &valor, &area1);
printf("Introduzca un caracter: ");
scanf("%c%*c", &letra);
```

Si ahora la entrada que introduce el usuario (marcada en negrita) es:

```
Introduzca un valor entero y otro real: 15 3.6
Introduzca un caracter: H
```

en la variable `valor` quedará almacenado el valor `15`, en `area1` el `3.6` y en `letra` el carácter `'H'`. El especificador de formato `%*c` lee el salto de línea (`'\n'`) y lo descarta. De esta manera, cuando se ejecuta la sentencia `scanf("%c%*c", &letra)`, el programa quedará a la espera de que el usuario introduzca un carácter, que posteriormente leerá y almacenará en la variable `letra`.

- La función `scanf()` espera del teclado el formato exacto especificado entre las comillas dobles. Si los datos no se introducen tal y como se indica, la función `scanf()` deja de leer del teclado y finaliza su ejecución, pero no la ejecución del programa. Por ejemplo, si tenemos el siguiente código:

```
int val1, val2;

printf("Introduzca dos valores enteros:");
scanf("%d-%d%*c", &val1, &val2);
```

y el usuario introduce desde el teclado `7,8` (en lugar de `7-8`), el programa lee el valor `7` y lo almacena en la variable `val1`; sin embargo, el valor `8` no lo lee porque el usuario no ha respetado el formato de entrada especificado en el primer parámetro (`%d-%d%*c`). Después de leer el `7`, la función `scanf()` finaliza su ejecución y el programa continuará ejecutándose con la variable `val1` valiendo `7` y la variable `val2` almacenando un valor indeterminado.

Nuestro primer programa en C

Ahora que ya conocemos la estructura básica de un programa, podemos escribir nuestro primer programa en C que muestra por pantalla un mensaje de texto.

```
#include <stdio.h>

main()
{
    printf("Has conseguido hacer tu primer programa en C.\n");
}
```

Algunas consideraciones sobre la escritura de los programas en C (normas de estilo):

- Las llaves (`{}`) del `main()` se abren y cierran en una línea exclusiva para ellas y están alineadas en la misma columna que el carácter `m` de `main()`.
- Todas las líneas de código dentro del `main()` (declaración de variables y sentencias) tienen un sangrado o indentación de 2 a 4 espacios respecto a las llaves del `main()`.
- Entre la declaración de variables y el conjunto de sentencias hay siempre una línea en blanco.